

Week 8: I2C Protocol & IMU Application - Notes

Table of contents

1	I2C Protocol Fundamentals	2
1.1	Overview	2
1.2	Key Characteristics	2
1.3	Open-Drain Signaling	3
1.4	I2C Timing Rules	3
1.5	Transaction Format	3
1.5.1	Address Byte	3
1.5.2	Write Transaction	3
1.5.3	Read Transaction	4
2	I2C Hardware Design	4
2.1	Pull-up Resistor Selection	4
2.1.1	Constraints	4
2.1.2	Practical Values	4
2.2	Bus Capacitance Sources	5
2.3	Multiple Pull-ups Problem	5
3	IMU Fundamentals	5
3.1	What is an IMU?	5
3.2	MEMS Accelerometer	6
3.2.1	Calculating Tilt	6
3.2.2	Limitations	6
3.3	MEMS Gyroscope	6
3.3.1	Characteristics	7
3.4	Sensor Fusion	7
4	MPU6050 Interface	7
4.1	Overview	7

4.2	I2C Address	7
4.3	Key Registers	8
4.4	GY-521 Chip Variants	8
4.4.1	MPU6050 vs MPU6500 at a Glance	8
4.5	Initialization Sequence	9
4.6	Data Conversion	9
4.7	Sensitivity Settings	10
5	Protocol Debugging	10
5.1	Common Issues and Solutions	10
5.2	Debugging Checklist	10
5.3	I2C Scanner	11
5.4	Bus Recovery	11
6	Quick Reference	11
6.1	STM32 HAL Functions	11
6.2	MPU6050 Quick Setup	12

1 I2C Protocol Fundamentals

1.1 Overview

I2C (Inter-Integrated Circuit) is a synchronous, multi-master, multi-slave serial communication protocol developed by Philips (now NXP) in 1982. It uses only two wires:

- **SDA (Serial Data):** Bidirectional data line
- **SCL (Serial Clock):** Clock signal generated by master

1.2 Key Characteristics

Feature	Description
Wires	2 (SDA + SCL) + ground
Topology	Multi-drop bus
Addressing	7-bit (128 devices) or 10-bit (1024 devices)
Direction	Half-duplex
Speed modes	Standard (100 kHz), Fast (400 kHz), Fast+ (1 MHz)
Driver type	Open-drain with external pull-ups

1.3 Open-Drain Signaling

I2C uses open-drain (or open-collector) outputs, meaning:

- Devices can only pull the line LOW (to ground)
- A pull-up resistor pulls the line HIGH when no device is driving
- This prevents bus contention (short circuits) when multiple devices communicate

Advantages:

- Multiple devices can share the same bus safely
- Any device can pull the line LOW without conflict
- Enables clock stretching (slave can hold SCL low to pause master)

1.4 I2C Timing Rules

1. **Data stability:** SDA must be stable when SCL is HIGH
2. **Data changes:** SDA can only change when SCL is LOW
3. **START condition:** SDA falls while SCL is HIGH
4. **STOP condition:** SDA rises while SCL is HIGH

1.5 Transaction Format

1.5.1 Address Byte

The first byte after START contains the 7-bit slave address and R/W bit:

A6	A5	A4	A3	A2	A1	A0	R/W
7-bit address							0=W
							1=R

1.5.2 Write Transaction

START -> Address+W -> ACK -> Register -> ACK -> Data -> ACK -> STOP

1.5.3 Read Transaction

START -> Address+W -> ACK -> Register -> ACK ->
Sr -> Address+R -> ACK -> Data -> NACK -> STOP

Where Sr = Repeated START (change direction without releasing bus)

2 I2C Hardware Design

2.1 Pull-up Resistor Selection

2.1.1 Constraints

Minimum resistance (current limit):

$$R_{min} = \frac{V_{CC} - V_{OL(max)}}{I_{OL}}$$

Where:

- $V_{OL(max)} = 0.4V$ (maximum output LOW voltage)
- $I_{OL} = 3 \text{ mA}$ (typical sink current)

$$R_{min} = \frac{3.3V - 0.4V}{3mA} = 967\Omega$$

Maximum resistance (rise time):

$$R_{max} = \frac{t_r}{0.8473 \times C_b}$$

Where:

- t_r = rise time (1000 ns for 100 kHz, 300 ns for 400 kHz)
- C_b = bus capacitance

2.1.2 Practical Values

Speed	Bus Capacitance	Recommended Pull-up
100 kHz	< 100 pF	4.7 kOhm
100 kHz	100-200 pF	2.2 kOhm
400 kHz	< 100 pF	2.2 kOhm

Rule of thumb: For breadboard prototyping at 100 kHz, 4.7 kOhm works well.

2.2 Bus Capacitance Sources

Source	Typical Value
Device input	3-10 pF
PCB trace (per inch)	1-2 pF
Breadboard (per row)	2-5 pF
Jumper wire (10 cm)	5-10 pF

2.3 Multiple Pull-ups Problem

When connecting multiple modules with onboard pull-ups:

$$R_{eff} = \frac{R_1 \times R_2}{R_1 + R_2}$$

Two 4.7 kOhm pull-ups in parallel = 2.35 kOhm (usually acceptable)

Watch for 3+ modules - may need to disable some module pull-ups.

3 IMU Fundamentals

3.1 What is an IMU?

An Inertial Measurement Unit (IMU) measures motion without external reference using:

- **Accelerometer:** Measures linear acceleration (including gravity)
- **Gyroscope:** Measures angular velocity (rotation rate)
- **Magnetometer:** Measures magnetic field (optional, for heading)

3.2 MEMS Accelerometer

Micro-Electro-Mechanical System (MEMS) accelerometers use:

- A proof mass suspended on springs
- Capacitive plates to detect displacement
- When accelerated, the proof mass moves relative to the housing

Key insight: At rest, the accelerometer measures gravity (1g downward).

3.2.1 Calculating Tilt

From accelerometer readings:

$$\theta_{pitch} = \arctan \left(\frac{A_x}{\sqrt{A_y^2 + A_z^2}} \right)$$

$$\theta_{roll} = \arctan \left(\frac{A_y}{\sqrt{A_x^2 + A_z^2}} \right)$$

3.2.2 Limitations

1. Cannot distinguish gravity from linear acceleration
2. High-frequency noise from vibration
3. Cannot measure yaw (rotation around gravity vector)

3.3 MEMS Gyroscope

Uses the Coriolis effect on a vibrating structure:

- A mass vibrates at a known frequency
- When rotated, Coriolis force causes perpendicular deflection
- Capacitive sensing measures this deflection

Output: Angular velocity in degrees per second (deg/s)

3.3.1 Characteristics

Strengths:

- Fast response to rotation
- Direct rotation measurement
- Immune to linear acceleration

Weaknesses:

- **Bias:** Non-zero reading when stationary
- **Drift:** Integration error accumulates over time

3.4 Sensor Fusion

Combining accelerometer and gyroscope compensates for individual weaknesses:

Complementary filter:

$$\theta = \alpha \times (\theta + \omega \times dt) + (1 - \alpha) \times \theta_{accel}$$

Where $\alpha \approx 0.98$:

- Trust gyroscope for short-term (fast, no drift in short term)
- Trust accelerometer for long-term (absolute reference to gravity)

4 MPU6050 Interface

4.1 Overview

The MPU6050 (InvenSense/TDK) is a popular 6-axis IMU:

- 3-axis accelerometer + 3-axis gyroscope
- I2C interface (up to 400 kHz)
- 16-bit ADC resolution
- Configurable measurement ranges
- On-chip Digital Motion Processor (DMP)

4.2 I2C Address

- **0x68** when AD0 pin is LOW (default)
- **0x69** when AD0 pin is HIGH

4.3 Key Registers

Address	Register	Purpose
0x6B	PWR_MGMT_1	Power management, clock, sleep mode
0x1B	GYRO_CONFIG	Gyroscope full-scale range
0x1C	ACCEL_CONFIG	Accelerometer full-scale range
0x3B-0x48	Sensor data	Accel, Temp, Gyro (14 bytes)
0x75	WHO_AM_I	Device ID (0x68 on MPU6050, 0x70 on MPU6500)

4.4 GY-521 Chip Variants

Boards sold as “GY-521 MPU6050” are not always genuine MPU6050 silicon. The `WHO_AM_I` register is the fastest way to tell:

Chip	WHO_AM_I	Notes
MPU6050	0x68	Original 2011 part
MPU6500	0x70	2014 revision — lower noise, faster, register-compatible for basic use
MPU9250	0x71	MPU6500 + AK8963 magnetometer
MPU9255	0x73	Upgraded MPU9250
ICM-20689	0x98	Drop-in replacement in some clones

For this lab, **MPU6050 and MPU6500 behave identically**: same `PWR_MGMT_1` (0x6B), same burst at 0x3B, same ± 2 g / ± 250 °/s default scales. Accept either ID in your `WHO_AM_I` check:

```
if (who == 0x68 || who == 0x70) { /* OK */ }
```

4.4.1 MPU6050 vs MPU6500 at a Glance

	MPU6050	MPU6500
Process	180 nm	90 nm
Supply	2.375–3.46 V	1.71–3.6 V

	MPU6050	MPU6500
Gyro noise	0.05 °/s/√Hz	0.01 °/s/√Hz
Accel noise	400 µg/√Hz	300 µg/√Hz
Max gyro ODR	8 kHz	32 kHz
I ² C max speed	400 kHz	1 MHz (Fast-mode+)
SPI	not supported	up to ~1 MHz (20 MHz for registers)
FIFO	1024 B	512 B
DMP	v2	v3

4.5 Initialization Sequence

1. Read WHO_AM_I (0x75) to verify device presence
2. Write 0x01 to PWR_MGMT_1 (0x6B) to wake up and select PLL clock
3. Wait 100ms for PLL to stabilize
4. Configure GYRO_CONFIG and ACCEL_CONFIG as needed

4.6 Data Conversion

Accelerometer (default +/-2g range):

$$A_g = \frac{A_{raw}}{16384}$$

Gyroscope (default +/-250 deg/s range):

$$\omega_{deg/s} = \frac{G_{raw}}{131}$$

Temperature:

$$T_C^{\text{MPU6050}} = \frac{T_{raw}}{340} + 36.53 \quad T_C^{\text{MPU6500}} = \frac{T_{raw}}{333.87} + 21.00$$

The temperature sensor is a **die thermometer**, not an ambient probe. Self-heating from the on-chip regulators and the gyro drive circuit typically puts the steady-state reading **5–10 °C above ambient**. Applying the MPU6050 formula to an MPU6500 (or vice-versa) shifts the result by roughly 15 °C — a common source of “my chip is running hot” reports.

4.7 Sensitivity Settings

Accel Range	Sensitivity
+/-2g	16384 LSB/g
+/-4g	8192 LSB/g
+/-8g	4096 LSB/g
+/-16g	2048 LSB/g

Gyro Range	Sensitivity
+/-250 deg/s	131 LSB/(deg/s)
+/-500 deg/s	65.5 LSB/(deg/s)
+/-1000 deg/s	32.8 LSB/(deg/s)
+/-2000 deg/s	16.4 LSB/(deg/s)

5 Protocol Debugging

5.1 Common Issues and Solutions

Symptom	Likely Cause	Solution
No ACK, lines HIGH	Missing pull-ups, wrong address	Check wiring, verify address
No ACK, SDA stuck LOW	Bus lockup	Clock recovery (9 pulses)
Garbage data	Speed too fast, wrong mode	Reduce speed, check timing
Intermittent failures	Loose connections	Check wiring, scope signals
WHO_AM_I returns 0x70	Board uses MPU6500, not MPU6050	Accept 0x70 as valid ID
WHO_AM_I returns 0x71/0x73	Board is MPU9250/MPU9255	Register-compatible for accel/gyro; accept ID

5.2 Debugging Checklist

1. **Power:** Is device powered correctly (3.3V or 5V)?
2. **Connections:** SDA to SDA, SCL to SCL, common ground?

3. **Pull-ups (powered check):** With the bus idle, SDA and SCL should sit near VCC (3.3 V). **Do not try to measure pull-up resistance with an ohmmeter from the VCC pin** — on GY-521 boards the pull-ups sit on the output of the onboard LDO, so you read through an off pass transistor and get hundreds of $k\Omega$ to $M\Omega$ regardless of whether the pull-ups exist.
4. **Address:** Correct 7-bit address? AD0 pin state?
5. **Software:** I2C peripheral enabled? Correct pin assignment?
6. **Chip ID:** Does WHO_AM_I match your silicon (0x68 for MPU6050, 0x70 for MPU6500)?

5.3 I2C Scanner

Use an I2C scanner to find all devices on the bus:

```
for (uint8_t addr = 1; addr < 128; addr++) {  
    if (HAL_I2C_IsDeviceReady(&hi2c1, addr << 1, 1, 10) == HAL_OK) {  
        printf("Device found at 0x%02X\n", addr);  
    }  
}
```

5.4 Bus Recovery

If SDA is stuck LOW (bus lockup):

1. Toggle SCL 9 times to clock out stuck byte
2. Generate STOP condition
3. If still stuck, power cycle the slave device

6 Quick Reference

6.1 STM32 HAL Functions

```
// Write to register  
HAL_I2C_Mem_Write(&hi2c1, addr << 1, reg, 1, &data, len, timeout);  
  
// Read from register  
HAL_I2C_Mem_Read(&hi2c1, addr << 1, reg, 1, buffer, len, timeout);
```

```
// Check if device ready
HAL_I2C_IsDeviceReady(&hi2c1, addr << 1, retries, timeout);
```

6.2 MPU6050 Quick Setup

```
#define MPU6050_ADDR  0x68
#define PWR_MGMT_1    0x6B
#define WHO_AM_I      0x75
#define ACCEL_XOUT_H   0x3B

// Wake up MPU6050
uint8_t data = 0x01;
HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR << 1, PWR_MGMT_1, 1, &data, 1, 100);

// Read 14 bytes of sensor data
uint8_t buffer[14];
HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR << 1, ACCEL_XOUT_H, 1, buffer, 14, 100);
```